

The Agentic Paradigm

What makes a system agentic and why it matters?

INTRODUCTION



In this course, we will try to answer:

How do we design AI systems that can
reason, plan, act, and coordinate
autonomously in real environments?

Before we can answer it, we need to understand what 'agentic' actually means

WHAT IS AGENTIC?

Agentic behaviour refers to systems that can decide what to do next (up to a certain level) rather than simply executing a predefined sequence

In other words,

goal-directed behaviour + *dynamic* decision making

Agentic systems exercise:

- **Autonomy:** chooses actions without a fixed script
- **Planning:** decomposes tasks into steps
- **Adaptivity:** reacts to intermediate outputs or failures
- **Tool use:** selects and invokes tools conditionally
- **Looping/reflection:** can revise its approach

WHAT IS AGENTIC?

AND WHAT IS NOT?

Three things that are commonly confused while talking about agentic systems



Pipeline

- A fixed sequence of steps
- Input goes in, output comes out (like RAG)
- No decisions made at runtime
- The logic is fully specified upfront; the system cannot deviate



Workflow

- A pipeline with branching
- Conditional logic is allowed, but every branch needs to be pre-specified
- The system follows a decision tree; it does not make decisions on its own



Agent

- A component that decides its next steps based on observation
- The control flow is determined at runtime by the model itself

Less ←
Autonomous

Semi-Autonomous

→ **More**
Autonomous

Pre-determined steps;
Prompt → response
without control

Some decisions;
autonomy to choose tools

Free to make decisions;
create new tools

PROPERTIES OF AN AGENTIC SYSTEM

A system must have all four properties. Missing even one means it is not truly agentic.

01

Goal-Direction

The system has an objective it is trying to achieve, instead of just responding to a prompt. It evaluates its own actions against that goal.

02

Iterative Reasoning

The system reasons across multiple steps. Each step informs the next. It doesn't produce a response in a single forward pass.

03

State

The system remembers what it has done and what it has learned across steps. Without state, every step starts from zero.

04

Consequential Actions

The system's actions change something: it calls an API, queries a database, writes a file. It is not just producing text.

AGENTIC SYSTEMS - CHECKLIST

Applying the checklist: are these systems agentic?

System	Goal-Dir.	Iterative	State	Actions
ChatGPT (basic Q&A)	X	X	X	X
ChatGPT with memory	X	X	✓	X
Basic RAG pipeline	X	X	X	✓
Tool-Caller	~	X	X	✓
GPT-4 browsing agent	✓	✓	✓	✓

✓ = present ~ = partial X = absent

Truly Agentic

WHY THIS DISTINCTION MATTERS?

Design decisions change completely

Pipelines are built top-down. Agents are built around a loop, state, and termination conditions.

Failure modes are different

Pipeline failures are deterministic and reproducible. Agent failures are probabilistic, context-dependent, and often subtle.

Evaluation is harder

Pipeline outputs can be checked against a ground truth. Agent evaluation requires measuring goal achievement, efficiency, and robustness across various aspects.

When NOT to use an agent

Most tasks do not need agents. A well-designed pipeline is faster, cheaper, and more reliable. Agents are appropriate when the task is long-horizon, unpredictable, or requires adaptive decision-making.

Agents and Tools

What are agents? Is tool use what makes a system agentic?

TOOLS AND AGENTS

A common misconception when starting with building agentic systems is

*I connected GPT-4 to a search tool and a calculator.
It's now an AI agent.*

Let's examine why it's wrong

What actually happened

The model was given a description of two tools

- It can choose to call them in a response
- It returns a response, but it does not loop
- It has no goal it is evaluating itself against
- There is no state across calls

What an agent would do differently

- Observe the result of each tool call
- Reason about whether the goal is closer
- Decide what to do next based on that
- Update its state before the next step
- Know when to stop

TOOLS vs AGENTS

A tool is a capability exposed as a callable function

It is *stateless, deterministic, external*, and ***does not make decisions***

Tool calling is a feature

An agent is a ***decision-making*** entity that can *select* and *use* tools to achieve a goal

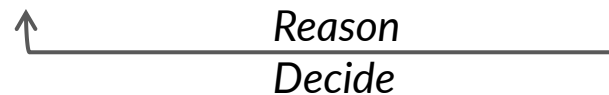
A tool does work and an agent decides what work is to be done

Agency is an architecture

An agent is not a simple tool call; It also combines autonomy and decision making

An agent is a closed-loop system that decides and executes actions, observes results, and maintains a state

$State_t \rightarrow Policy (LLM) \rightarrow Action (tool) \rightarrow Environment \rightarrow Observation \rightarrow State_{t+1}$

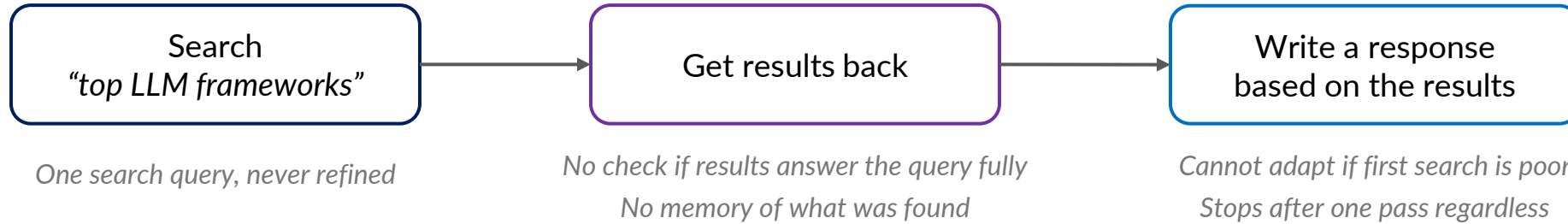


↻ loops until goal is achieved or termination condition is met

EXAMPLE

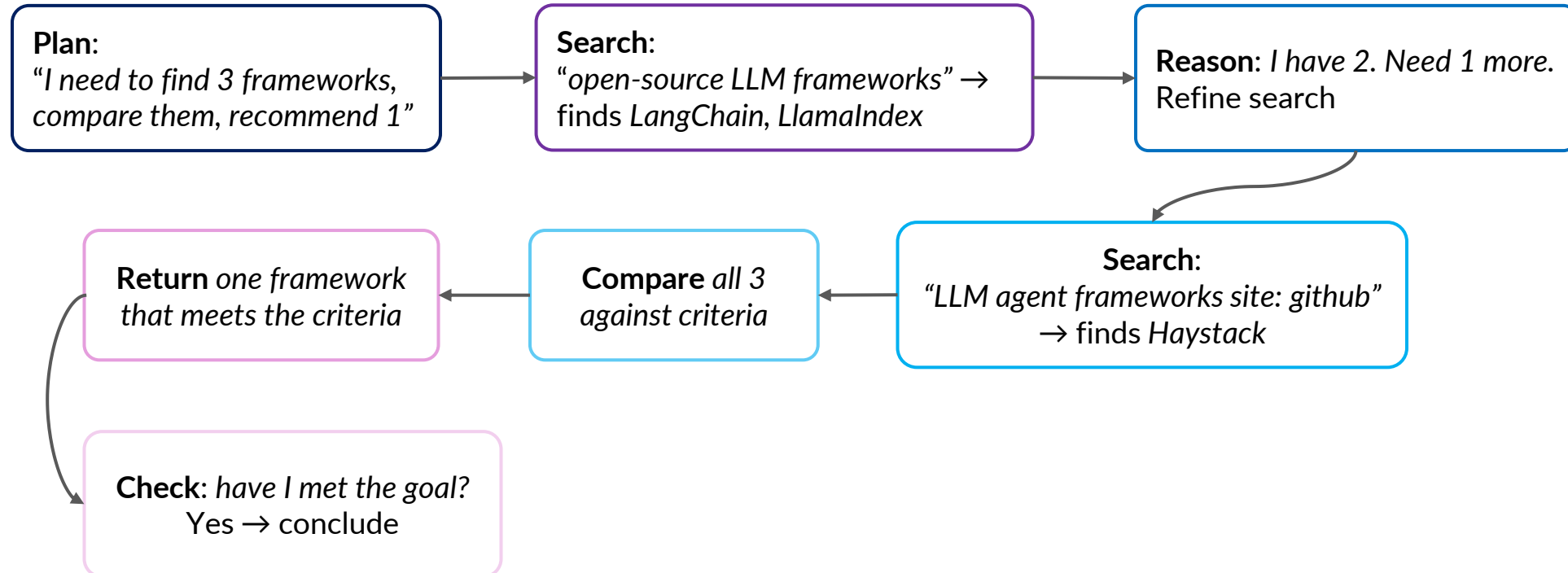
Task: "Research the top 3 open-source LLM frameworks and recommend one for a beginner."

Tool-calling
(non-agentic)



Agentic approach

adaptive,
goal-directed,
verifiable



AGENTIC SYSTEMS

An agentic system is any system that exhibits or enables agentic behaviour, whether through explicit agents or through architectures that approximate agency

An agentic system is a superset architecture which may include:

Agentic Systems

— Agent(s)	← <i>decision-makers</i>
— Tools	← <i>capabilities</i>
— Memory	← <i>state persistence</i>
— Environment	
— Orchestration	(<i>optional constraints</i>)

Agents do not replace a tool. What changes is the architecture around the tool.
An agent uses tools inside a loop, with state, towards a goal

Termination, Self-Consistency, and Reflection

STOPPING AGENT LOOPS

Loops need exits; Termination is an important design decision in an agent loop, not an edge case

Every termination condition must be explicitly programmed

An agent loop is a *while* loop; Like all while loops, it runs indefinitely unless something breaks the cycle

Unlike a web server or a data pipeline (where infinite loops are just bugs), an agentic loop that never terminates **costs money**, **burns context window**, and **produces no output**

Without an explicit termination strategy:

- The agent calls tools in circles, never deciding it's done
- It fills the context window with redundant observations
- It produces a response only when forced to by a token limit
- That response is rarely the one you wanted

TERMINATION STRATEGIES

Step cap

- Hard limit on the number of loop iterations
- Always present
- The agent stops regardless of whether the goal is achieved

Use when: Always

Goal check

- After each step, the agent evaluates whether the goal has been met
- Uses the LLM itself as the judge

Use when: When goals are well-specified and verifiable

External signal

- A human, a test, or an external system confirms completion
- The agent pauses and waits

Use when: High-stakes decisions, human-in-the-loop workflows

Use at least one. Use all three in production.

A step cap is necessary but not sufficient. A goal check is sufficient but not safe alone.

STEP CAP

Set it always; normally we use this as an emergency brake

How it works:

1. Before each loop iteration, check whether the step counter has reached the maximum
2. If yes, stop (*regardless of whether the agent has achieved the goal*)
3. Emit whatever partial answer exists, or raise an error

```
while not state.done:
    if state.step_count >= state.max_steps:
        state.done = True
        break
    ...
```

- Always include a step cap, even when you have a goal check
- Typical range: 5–15 steps for most tasks; 50+ for long workflows
- Instead of steps, we can also limit the execution time or tool calls

A very low max_steps can cause the agent to fail valid tasks. Too high and errors become expensive.

GOAL CHECK

After each step, the agent asks: *Is the goal achieved?*

Without goal check

- Agent runs until step cap is hit
- Stops mid-task on step 10 even if step 7 had the answer
- No way to distinguish 'done' from 'gave up'
- Output quality is unpredictable

With goal check

- After each action, agent re-reads the goal
- Asks itself: '*Is the original question fully answered?*'
- If yes → emit final answer, set *done* = *True*
- If no → continue to next step

- This can also be done using the *state*, if no new information is retrieved and observations are repeated, the loop is terminated
- The LLM does the work as well as evaluates whether the work is complete

If goal check over-triggers 'good enough', the loop may stop too early;

If it returns 'not done' always, the loop never stops

SELF CONSISTENCY

Self-consistency is a technique where multiple independent reasoning paths are generated, and the final answer is selected based on majority/agreement

1. Generate N responses

Run the same prompt through the agent N independent times
Each run reasons from scratch

2. Compare the answers

Extract the final answer from each run
Group identical or semantically equivalent answers

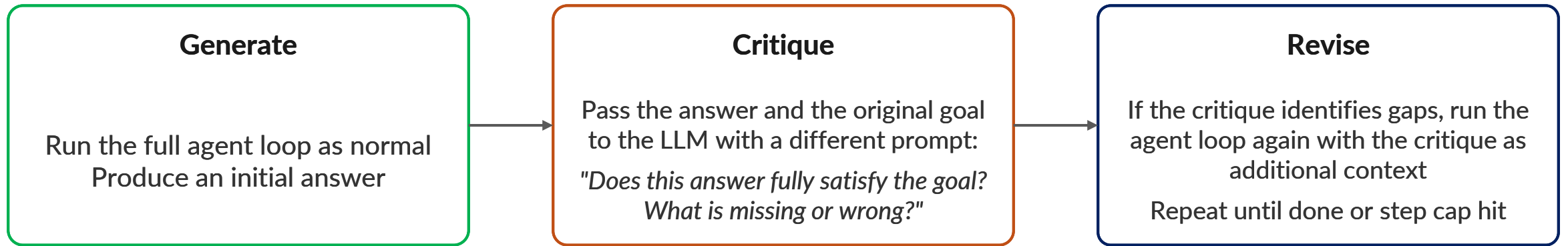
3. Return the majority

The answer that appears most frequently is the output
Outliers are discarded

- Useful because LLMs are stochastic and sensitive to sampling
- Aggregation can be done based on majority voting, confidence-weighted selection etc.
- Cost: $N \times \text{tokens} \times \text{API calls}$
- Use when: a single run is unreliable and the task has a verifiable answer, like math, reasoning etc.

REFLECTION

Reflection is the process where an agent evaluates and improves its own output
The agent evaluates its own answer against the original goal



Reflection loops need their own termination condition, e.g.:

*A critique step that always finds something wrong creates an infinite revision cycle.
Cap reflection passes at 1–3 iterations.*

TYPES OF REFLECTION

01

Output Reflection

- Check correctness of final answer
- Fix errors

02

Process Reflection

- Evaluate reasoning steps
- Identify flawed assumptions

03

Tool-Use Reflection

- Did the agent choose the right tool?
- Was the query appropriate?

04

Memory Reflection

- Store insights for future tasks
- Update long-term memory

USING THESE TECHNIQUES

Start with a step cap. Add goal check for production. Add the rest only if the task demands it.

Technique	Use when	Cost overhead	Main risk
Step cap	Always, every agent	None	Stops valid tasks if set too low
Goal check	Tasks with a clear success criterion	1 LLM call / step	Over-triggers; declares done too early
Self-consistency	Math, reasoning, factual queries	$N \times \text{tokens}$	Expensive; not useful for open-ended tasks
Reflection	High-stakes outputs, written content	1 to $3 \times \text{tokens}$	Infinite revision loop without a cap

These mechanisms are often combined, as and when needed, in the following order:

```
while not terminated:
    think
    act
    observe

    if reflection enabled:
        critique + revise

    if self-consistency enabled:
        sample alternatives + compare
```

Tool Schema, Validation, And Agent Skills

TOOL SCHEMA

A tool schema is a formal contract that specifies inputs (types, structure, constraints), outputs (expected structure), and semantics (what the tool does)

How you describe a tool determines whether the LLM uses it correctly

A tool schema has four parts:

Name	What the LLM calls the tool <i>search_web, calculate_value, read_file</i>
Description	When, why, and what NOT to use the tool for This is read by the LLM and treated as an instruction
Input schema	JSON Schema for arguments. Use <i>enums</i> to constrain categorical inputs. Set <i>maxLength</i> on strings. Mark required fields.
Output (implicit)	Document what the tool returns in the description The LLM must know what format to expect

A tool should return deterministic outputs
Vague descriptions produce hallucinated calls

DEFINING A GOOD SCHEMA

X Bad schema

name: *lookup*
description: *'Look something up.'*
input: { *query: string* }

Problems:

- No guidance on when to use
- No constraints on input
- No output format described
- Category of search undefined

✓ Good schema

name: *search_company_info*
description:
'Search for founding year, release year, HQ, use case. Returns one sentence. Do NOT use for financial data.'
input: {
 query: string (max 200 chars),
 category:
 enum[founding|release|location|use_case] }

CORE DESIGN PRINCIPLES

01

Strong typing (avoid ambiguity)

```
{"k": {  
  "type": "integer",  
  "description": "total fetched results"  
  "minimum": 1,  
  "maximum": 20 }}
```

02

Explicit constraints

```
# constrain the action space  
{ "operation": {  
  "type": "string",  
  "enum": ["sum", "mean", "max"]  
}}
```

03

Semantic clarity

Descriptions must encode when to use the tool and when not to use it.
This directly affects agent decision quality.

04

Minimal surface area

Avoid overloaded tools (do everything) and excessive optional parameters.
Prefer multiple specialised tools.

VALIDATION

Validate inputs, validate outputs

A tool should fail fast with a clear error, not quietly corrupt downstream reasoning

Input validation (Syntactic)

Before calling

- Validate *tool_input* against the JSON Schema before executing
- Catches type errors, missing fields, invalid enumerations
- Use `json_schema.validate()`

Output validation

After calling

- Check tool use, output format before returning to the agent
- Empty string, error prefix, or suspiciously short result should be flagged and handled

Error as tool result

On failure

- Return the error as a string *tool_result*
- The LLM can reason about it
- Never raise an exception that crashes the loop

Validation should be **external to the agent**

AGENT SKILLS

- A skill is a reusable, composable capability that encapsulates a multi-step behaviour, potentially involving reasoning, tools, and control flow
- Skills
 - Are reusable across tasks
 - Are composable with other skills
 - Work at a higher-level than tools
 - May internally use tool(s), sub-agents, intermediate state, or reasoning steps
 - Skills enable hierarchical design, reuse, and abstraction

*Tools define
capabilities*

```
search(query)  
summarise(text)
```

*Skills define
strategies*

```
research_topic(topic):  
    → search multiple sources  
    → filter relevant docs  
    → summarise  
    → synthesise answer
```

TOOLS vs SKILLS

Tool

An external capability the agent can invoke

Examples:

- `web_search` (query)
- `read_file` (path)
- `run_sql` (query)
- `send_email` (to, body)

A tool is atomic; it does one specific thing and returns a result

Skill

A reusable, composable capability that an agent exposes which other agents can invoke

Examples:

- `research_topic` (question) → summary
- `summarise_document` (doc) → brief
- `analyse_sentiment` (text) → score

A skill is a packaged agent behaviour; tools + logic wrapped as a callable unit

In multi-agent systems, agents consume tools from the environment and expose skills to other agents

COMPOSITION

Skills can be composed into larger skills, for example

Primitive tools

`search() · summarize() · store() · retrieve()`

Skill: FrameworkResearchSkill

`run(framework) → {name, summary, key}`

Uses search → summarize → store, internally

Skill: FrameworkComparisonSkill

`run([frameworks]) → comparison string`

Uses: FrameworkResearchSkill × 3

Skills can be:

- **Sequential:** *Skill A → Skill B → Skill C*
- **Conditional:** *If X → Skill A; Else → Skill B*
- **Hierarchical:** *Skill = subskills + tools*

Agent Failure Modes

FAILURE MODES

Generally, we see the following five failure modes, in order of frequency

- 1 Hallucination loop**

Fabricates tool results or reasoning. Fills gaps with plausible content.

- 2 Infinite retries**

Same failing call, repeated. Hits step cap without progress.

- 3 Shallow reasoning**

Stops at first plausible result. Doesn't verify completeness.

- 4 Goal drift**

Loses track of original objective. Answers the wrong question.

- 5 Knowledge mismatch**

Uses stale training data where current retrieval was required.

Agents fail differently from traditional software, confident and well-formatted

Agent failures leave signals in the trace, we need observability

HALLUCINATION LOOP

- The agent repeatedly generates and reinforces incorrect intermediate beliefs, treating them as valid observations
- It uses an incorrect initial hypothesis as the input for next steps, result in error compounding across iterations
- It is caused mainly due to lack of grounding or weak validation

What it looks like

Tool returns: *'No data available'*

Agent says: *'Based on current prices, AAPL is trading at \$182 and shows bullish momentum...'*

- Confident but incorrect chains of reasoning
- Repeated justification of the same false claim
- No tool usage when tools are available
- Fabricated intermediate “facts”

How to detect & fix

Detection: compare agent's stated facts against actual *tool_results* in the trace.

Fixes:

- Explicit no-data handling in system prompt
- Force grounded answers by adding constraints
- Disable reasoning and force tools/retrieval
- Counterfactual prompting: *'what evidence contradicts your conclusion'*

INFINITE RETRIES

- The agent fails to converge and continues iterating without reaching a stopping condition
- Weak termination criteria and reflection loops keep triggering retries
- System prompt says 'retry' but not 'with a different approach'

Symptoms

Tool fails. Agent calls same tool with same arguments. Fails again. Calls again

- Repeated tool calls with similar inputs
- Cyclic reasoning (“try again” patterns)
- No progress in state
- No convergence
- Over-aggressive reflection

Detecting and Fixing

- Track (tool, input) pairs
- Loop trace analysis, track step counts
- Monitor state changes and add rules to stop (less change → no progress)
- Block identical retries after N attempts
- Inject: ‘If a tool fails twice with the same input, try a different approach’

SHALLOW REASONING

- The agent produces answers without sufficient decomposition, exploration, or verification
- It skips planning and uses the first plausible answer, even if goal requires further actions

Symptoms

- Correct format, weak substance
- Missing intermediate steps
- Fails on compositional tasks
- For example, immediately responding after getting the first search result, even if the goal requires five data points

Detecting and Fixing

- Slightly modify inputs and observe output perturbations; drastic output changes show fragile reasoning
- Compare multiple samples and check diversity; outputs should be more diverse
- Depth-probing in prompt: *'Answer EVERY part of the question for EVERY item requested'*
- Enforce planning and self-consistency

GOAL DRIFT

- The agent deviates from the original objective during multi-step execution
- Intermediate steps redefine the task and subgoals get prioritised over main goal
- Context accumulation over multiple steps causes dilution

Symptoms

- Output is coherent but irrelevant
- Task partially completed or misinterpreted
- Excessive focus on side issues

Detecting and Fixing

- Check goal alignment at each step
- Map which part of the goal each step satisfies
- Re-inject the goal at every step
- Use persistent goal memory

KNOWLEDGE MISMATCH

- The agent's internal knowledge is outdated, incomplete, or inconsistent with the environment
- The model relies on parametric knowledge and has no grounding
- Absence of retrieval layer or ignoring tool results can lead to knowledge mismatches

Symptoms

- Confident but outdated answers
- Incorrect factual references
- Misalignment with database/tool outputs

Detecting and Fixing

- Force retrieval before reasoning
- Compare agent answer with tool outputs
- Force usage of retrieved data
- Check for source attribution

DIAGNOSTIC CHECKLIST

1 Read the trace

What tools were called? What did each return? Does the reasoning match actual results?

2 Hallucination check

Did the agent claim any fact not present in a tool result? Did it interpret 'no data' as data?

3 Retry check

Are any (tool, input) pairs repeated more than twice?

4 Coverage check

Did the agent address every part of the goal? Would a human notice anything missing?

5 Goal check

Does the final answer address the original question, not a drifted version of it?

6 Memory check

Does the answer include time-sensitive facts not backed by a tool result?